

Preamble. This lab deals with several interval-reduction methods for finding the minimum of unimodal functions on an interval $[a, b]$. The notion of a unimodal function was introduced in the lecture notes on nonlinear optimization. All implementations requested in this lab are to be done in **Julia**.

Interval-reduction methods The principle behind interval-reduction methods for finding the minimum x^* of a unimodal function f is to shrink, at each iteration k , the interval $[a_k, b_k]$ in which x^* lies and on which f remains unimodal (taking $a_1 = a$ and $b_1 = b$). When this interval becomes small enough (for instance, once its length $(b_k - a_k)$ is smaller than 10^{-7}), convergence is considered to be reached, and the solution x^* can be taken as either endpoint of the interval.

1 Bisection search method

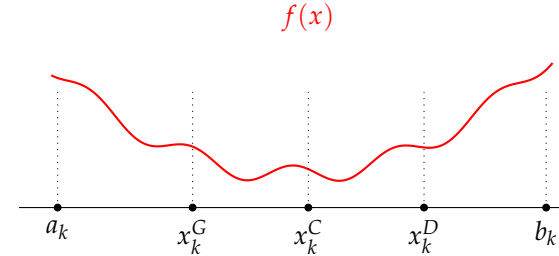
As its name indicates, the bisection search method halves the interval $[a, b]$ at every iteration. Let $[a_k, b_k]$ be the interval obtained after the first $k - 1$ iterations. At each iteration k , three equally spaced points are computed in the interval $[a_k, b_k]$:

$$x_k^G = a_k + \frac{1}{4}(b_k - a_k)$$

$$x_k^C = a_k + \frac{1}{2}(b_k - a_k)$$

$$x_k^D = a_k + \frac{3}{4}(b_k - a_k)$$

We also evaluate $f(x_k^G)$, $f(x_k^C)$ and $f(x_k^D)$.



The interval $[a_k, b_k]$ is reduced according to the following 3 cases:

1. if $f(x_k^C) > f(x_k^D)$ then $a_{k+1} = x_k^C$ and $b_{k+1} = b_k$. Note that $x_{k+1}^C = x_k^D$.
2. if $f(x_k^C) > f(x_k^G)$ then $a_{k+1} = a_k$ and $b_{k+1} = x_k^C$. Note that $x_{k+1}^C = x_k^G$.
3. otherwise, i.e. if $f(x_k^C) \leq f(x_k^D)$ and $f(x_k^C) \leq f(x_k^G)$, then $a_{k+1} = x_k^G$ and $b_{k+1} = x_k^D$. Note that $x_{k+1}^C = x_k^C$.

In every case, x_{k+1}^C can be taken among the points x_k^G , x_k^C and x_k^D . We therefore only need to compute and evaluate x_{k+1}^G and x_{k+1}^D . The bisection search method thus requires 2 evaluations of the function f at each iteration, except for the first one.

Q1. For each of the cases above, show that x^* lies in the new interval and that f remains unimodal on it.

Q2. Implement the bisection search method in Julia and check that it gives the expected result for the following four functions on the interval $[-1000, 1000]$.

$$f_1(x) = |x - 100|$$

$$f_2(x) = \begin{cases} \sqrt{x - 50} & \text{if } x \geq 50 \\ \sqrt{-(x - 50)} & \text{if } x \leq 50 \end{cases}$$

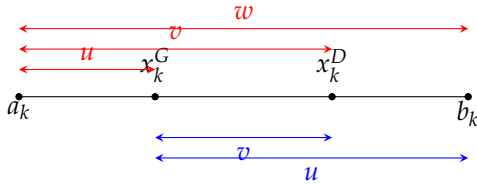
$$f_3(x) = \min(4x, x + 5)$$

$$f_4(x) = -x^3$$

2 Golden section method

In terms of the complexity of interval-reduction methods, the total number of evaluations of the function f is a factor that can matter even more than the number of iterations. It would therefore be useful to have a method that requires only a single evaluation of f per iteration instead of two, even at the cost of reducing the interval less. The golden section method meets this goal, as we shall see in this part.

The bisection search method can be adapted by computing only 2 points x_k^G and x_k^D in the interval $[a_k, b_k]$, such that $a_k < x_k^G < x_k^D < b_k$. Throughout this part we assume that the distance u between a_k and x_k^G is the same as the distance between x_k^D and b_k , as illustrated in the figure below.



The interval $[a_k, b_k]$ is then reduced according to the following cases:

1. if $f(x_k^G) > f(x_k^D)$ then $a_{k+1} = x_k^G, b_{k+1} = b_k$
2. if $f(x_k^G) < f(x_k^D)$ then $a_{k+1} = a_k$ and $b_{k+1} = x_k^D$
3. otherwise, i.e. if $f(x_k^G) = f(x_k^D)$, then $a_{k+1} = x_k^G$ and $b_{k+1} = x_k^D$.

In the first two cases, the interval is divided by a factor

$$\alpha = \frac{w}{v}$$

Note that the third case is very unlikely.

Q3. To begin, show that x^* lies in the new interval and that f remains uni-modal on it.

Q4. Express x_k^G and x_k^D as functions of a_k, b_k and the factor α .

Q5. Suppose, for now, that the points are equally spaced as in the figure below. By what factor is the interval divided at each iteration, for each case?



Q6. How many evaluations of the function f must be carried out at each iteration?

Q7. How should x_k^G and x_k^D be positioned in order to reduce the interval as much as possible at each iteration, for the first two cases?

The golden section method consists in choosing the points x_k^G and x_k^D so that one of these points is reused at the next iteration. There will then be only a single evaluation of f per iteration.

More precisely, we would like to have:

1. if $f(x_k^G) > f(x_k^D)$, then $x_{k+1}^G = x_k^D$
2. if $f(x_k^G) < f(x_k^D)$ then $x_{k+1}^D = x_k^G$
3. if $f(x_k^G) = f(x_k^D)$, we accept evaluating the function twice, since this happens only very rarely

To have $x_{k+1}^G = x_k^D$ in the first case and $x_{k+1}^D = x_k^G$ in the second case, it suffices to have

$$\alpha = \frac{w}{v} = \frac{v}{u}$$

Q8. Determine the numerical value of α satisfying the previous equation.

Q9. By what factor is the interval reduced at each iteration for the golden section method, in each case?

Q10. How many evaluations of the function f must be carried out at each iteration for the golden section method?

Q11. Implement the golden section method in Julia and check that it gives the expected result for the functions f_1, f_2, f_3, f_4 .

3 Application: subscription problem with penalty

Consider the subscription problem for a regularly consumed commodity (water, electricity, gas, internet, ...). We focus in particular on a parameter that is the maximum power draw at any given time. For example, for electricity, a household typically subscribes to a maximum meter power of 6kVA. If, at some point in time, consumption exceeds this power, the meter trips, which causes a power outage. To avoid this outage (which can be especially costly, in particular for businesses), an alternative type of subscription is offered, in which consumption is allowed to continue, but the customer must pay a penalty for exceeding the subscribed capacity. For this type of contract, instead of being assigned a fixed maximum subscribed power, the customer is free to choose it. The problem that then arises for a customer is to choose an optimal subscribed power based on their consumption habits. In this lab, we consider a simplified version of this problem. Given a consumption record in the form of a vector $L = \{c_1, c_2, \dots, c_n\}$, where c_i denotes the consumption during period i . The cost the customer would have to pay for their consumption is defined by the following function f

$$f(p) = a * p + b * \sqrt{\sum_{i=1}^n (\max(0, c_i - p))^2}$$

where p is the subscribed power. The first part of $f(p)$ is $a * p$, where a is a positive constant representing the fixed cost of the subscription. The second part, $b * \sqrt{\sum_{i=1}^n (\max(0, c_i - p))^2}$, represents the cost of the capacity-overrun penalty. This cost is computed as the 2-norm of the overrun vector, multiplied by a positive constant b . Let $C = \max\{c \mid c \in L\}$ be the largest consumption value. The optimal subscribed-power problem can then be formulated mathematically as

$$\min_{0 \leq p \leq C} f(p)$$

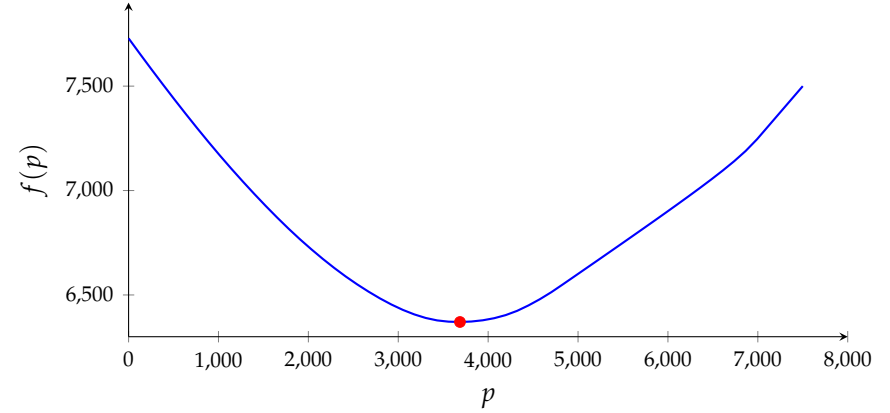


Figure 1: Cost function for the subscription problem with penalty

The fixed-cost part of $f(p)$ is linear and strictly increasing, and the penalty part is (convex and) strictly decreasing. It can be shown that $f(p)$, being the sum of these two parts, is unimodal on the interval $[0, C]$. A rigorous proof is beyond the scope of this lab, but one can intuitively picture the unimodality of f as follows: there is a first, decreasing phase, in which, for small values of p , the penalty part dominates the fixed-cost part. The function becomes increasing once this dominance reverses, for large values of p . The value p^* that minimizes $f(p)$ is exactly the point at which the two parts are in balance.

Q12. Write a Julia function that takes as parameters p , L , a and b and computes the value of $f(p)$ using the parameters L , a and b .

Q13. Write a Julia function to plot f on the interval $[0, C]$ (for instance using the `Plots.jl` package). For example, Figure 1 shows the function f plotted with $a = 1$, $b = 0.5$ and

$$L = [2500, 3500, 4380, 4389, 4725, 4800, 3700, 3500, 7000, 7500, 2000, 1200].$$

Q14. Write a Julia function that finds the minimum p^* of $f(p)$ on the interval $[0, C]$ using the bisection search method.

Q15. Write a Julia function that finds the minimum p^* of $f(p)$ on the interval $[0, C]$ using the golden section method.

Q16. Compare the efficiency of the two methods in terms of running time and total number of evaluations of f , by testing them on 100 randomly generated instances with the same length for the vector L and the same constants a and b .